

WEBSITE SCRAPER

Information Assurance & Security

TEAM MEMBERS:

Farah Haddad
Mariem Maddouri
Zeineb jaghmoun
Mayssa Ben yousef
Mariem Soltani

PRESENTATION ROADMAP

1. Foundations & Core Paradigms

2. Architecture & System Demo

3. Implementation Stack & Lifecycle

4. Threat Modeling & Mitigations

INTRODUCTION

DATA AS A CORE ASSET:

Modern Business Analytics and IT operations rely on real-time external data (market intelligence, competitor pricing, threat feeds).

THE SCALE INVERSION:

The public web functions as the world's largest open-source repository. However, manual extraction is completely unviable.

THE SECURITY FRICTION POINT:

Standard automated harvesting scripts frequently execute uncontrolled, high-frequency loops. This turns benign scripts into unintentional distributed resource exhaustion vectors, threatening target server uptime and violating data boundary protocols.



1. Foundations & Core Paradigms

PROBLEM STATEMENT

Traditional manual data collection from websites is extremely time-consuming and error-prone. Teams spend hours copying data that could be automated.

Need for Automation

Businesses require scalable, efficient solutions to gather web data automatically. Manual processes cannot keep pace with modern data demands.



Manual Collection

Time-intensive process prone to human errors and inconsistencies.



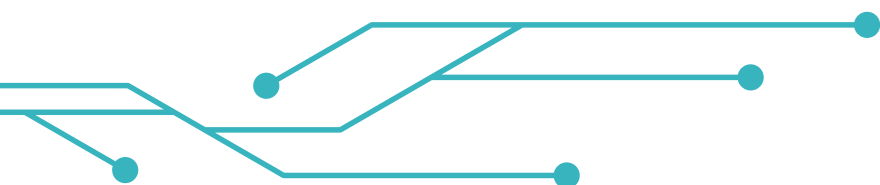
Unstructured Data

Raw web data lacks organization, making analysis difficult.



Scalability Issues

Manual methods fail to handle large-scale data requirements.



PROJECT OBJECTIVES

Replace time-consuming manual data collection with efficient automated web scraping processes that save hours of repetitive work.

Structured Output

Transform unstructured web content into clean, organized data formats (JSON, CSV, databases) ready for analysis and integration.

Automation

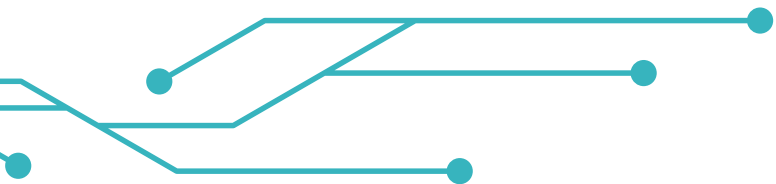
Eliminate manual acquisition workflows by deploying a programmatic pipeline capable of harvesting targeted web content at scale.

Data Structuring & Normalization

Transform flat, unorganized HTML layout files into clean, structured, and schema-consistent datasets ready for analytics.

Ethical Scraping & Boundary Governance

Enforce automated policy checks that programmatically parse and strictly obey host server access rules (robots.txt) and crawl delays.



1. Foundations & Core Paradigms

PROPOSED SOLUTION

A Decoupled Web Application and Scraping Architecture that Merges Automated Data Extraction with Strict Ethical Perimeter Controls.

Split Architecture

The system isolates user interactions from raw network mechanics by splitting the application into a web-based Flask user interface and an independent Python extraction engine.

Mandatory Security Gatekeeper

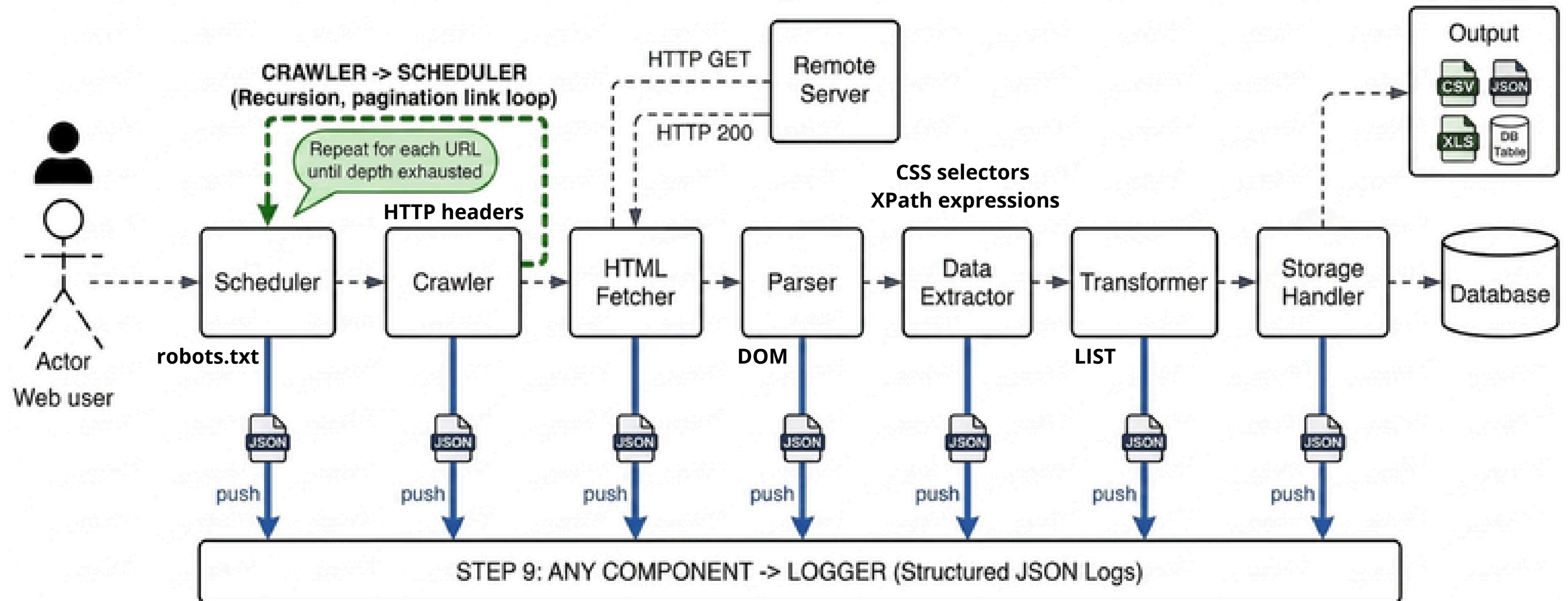
Ethical compliance is built directly into our core pipeline. Before downloading any data, the system automatically checks the website's rules for permission and applies a mandatory time delay to protect the host server from overloading.

Resilient Error Capture

The system includes built-in error handling to safely capture network timeouts, expired certificates, and server blocks, transforming fatal backend crashes into clear alerts on the user dashboard.

2.Architecture & System Demo

WORKFLOW PIPELINE



Each stage processes and refines data, ensuring clean, usable output.

2.Architecture & System Demo

step 1 : User → Scheduler.

The user submits a JSON configuration object containing the **target URL**, the **list of fields to extract**, and the desired **format**.
This is the entry point that initialises the entire scraping job.

Step 3 : Crawler → HTML Fetcher.

The Crawler passes the target URL along with **HTTP headers** (including a realistic **User-Agent** string and any required cookies) to the HTML Fetcher, triggering an **HTTP GET request** to the remote server.

Step 2 : Scheduler → Crawler.

After validating the configuration and checking **robots.txt**, the Scheduler packages the task into a task object and pushes it onto the **internal job queue**.
The Crawler picks it up and begins navigation.

Step 4 : HTML Fetcher → Parser

The HTML Fetcher receives **the HTTP 200 response** body and forwards **the raw HTML string** to the Parser for structural analysis.

Step 5 : Parser → Data Extractor.

The Parser constructs a **Document Object Model** (DOM) from the raw HTML and passes the resulting **structured node tree** to the Data Extractor.

Step 6 : Data Extractor → Transformer.

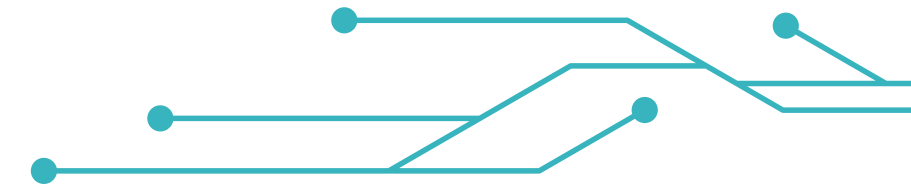
Using **CSS selectors** or **XPath expressions** defined in the configuration, the Data Extractor pulls the required field values (strings, attributes, links) and delivers them as a **list of key-value pairs** to the Transformer.

Step 7 : Transformer → Storage Handler.

The Transformer cleans and normalises the raw fields : **trimming whitespace, converting data types, Remove empty values** ,and **Truncate very long strings** then passes a clean, typed **dict list** to the Storage Handler.

Step 8 : Storage Handler → Output

The Storage Handler writes the final dataset to the chosen output format: a **CSV** file, a **JSON** file, an **Excel** (.xlsx) workbook



Any Component → Logger

Throughout all steps, every component emits structured log entries to the centralized logger, recording request outcomes, warnings, errors, and performance metrics for audit and debugging purposes.

Crawler → Scheduler (Recursion)

When the Crawler identifies next-page links (pagination or infinite scroll), it pushes the newly discovered URLs back to the Scheduler's queue. Steps 2 through 8 are then repeated for each URL until the configured crawl depth is exhausted or no further pages are found.



LIVE DEMO

USER WORKFLOW



Enter URL

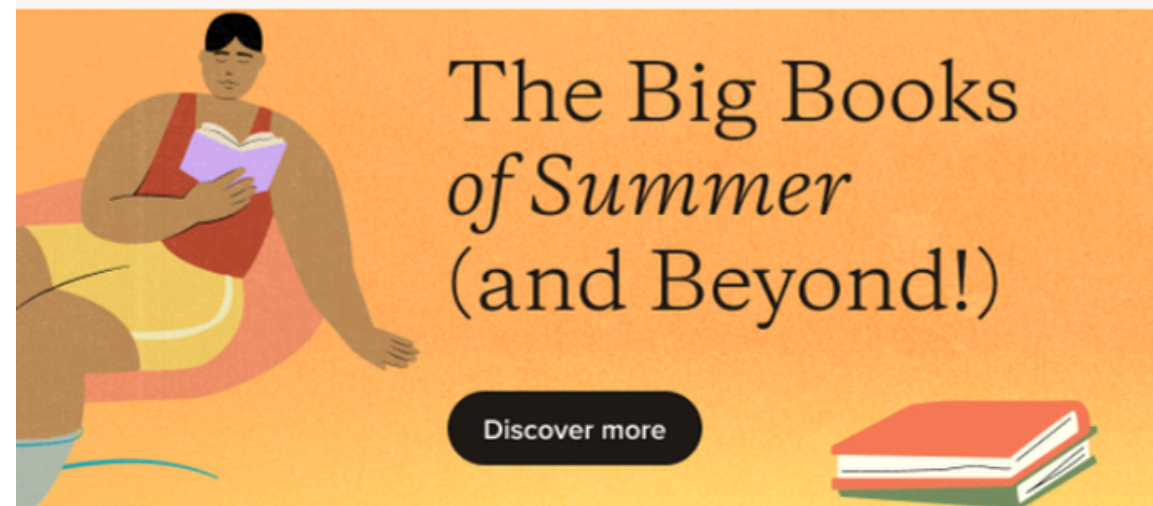


Select Data Fields



Download Results

goodreads



Deciding what to read next?

You're in the right place. Tell us what titles or genres you've enjoyed in the past, and we'll give you surprisingly insightful recommendations.

What are your friends reading?

Chances are your friends are discussing their favorite (and least favorite) books on Goodreads.

What will you discover?

Because Shomeret liked...



Nonfiction,
Microhistory

WS

WEBSITE SCRAPER

Cybersecurity Project · Information Assurance and Security · TBS 2025/2026

// 01 - TARGET URL

Enter the website URL to scrape

https://example.com

// 02 - DATA TO EXTRACT

Choose what you want to collect from the page

☐ Page Title

☐ Headings (H1-H3)

☐ Paragraphs

☐ All Links

☐ Images

☐ Meta Description

☐ All Text

// 03 - OUTPUT FORMAT

Save results as

CSV (spreadsheet-friendly)

▶ RUN SCRAPER

WS

WEBSITE SCRAPER

Cybersecurity Project - Information Assurance and Security - TBS 2025/2026

// 01 - TARGET URL

Enter the website URL to scrape

`https://www.goodreads.com`

// 02 - DATA TO EXTRACT

Choose what you want to collect from the page

☐ Page Title

☐ Headings (H1-H3)

☐ Paragraphs

☐ All Links

☐ Images

☐ Meta Description

☐ All Text

// 03 - OUTPUT FORMAT

Save results as

CSV (spreadsheet-friendly)

OUTPUT

● Done! Extracted 150 records.

// SECURITY & COMPLIANCE REPORT

✓ robots.txt Allowed

✓ TLS Verified

✓ User-Agent Rotation

✓ Rate Limiting Active

robots.txt allows scraping this path.

// SCRAPE SUMMARY

200

HTTP Status

150

Records Found

349

Response (ms)

! Download Output File

	A	B
1	field	value
2	title	Goodreads Meet your next favorite book
3	h2	Discover & read more
4	h2	Deciding what to read next?
5	h2	What are your friends reading?
6	h2	What will you discover?
7	h2	Search and browse books
8	h2	Quotes
9	h2	Goodreads Choice Awards: Readers' Favorite Books 2023
10	h3	News & Interviews
11	h2	Love lists?
12	h2	Are you an author or a publisher?
13	h3	Company
14	h3	Work with us
15	h3	Connect
16	link	/
17	link	https://www.goodreads.com/ap/signin?identityProvider=
18	link	https://www.goodreads.com/ap/signin?identityProvider=
19	link	https://www.goodreads.com/ap/register?language=en-US
20	link	/about/terms
21	link	/about/privacy
22	link	/user/sign_in
23	link	https://www.goodreads.com/blog/show/3088?ref=bookshelves
24	link	/book/show/56769575-stiff
25	link	/book/show/6493208-the-immortal-life-of-henrietta
26	link	/book/show/55338982-cultish
27	link	/book/show/208430625-vanishing-treasures
28	link	/book/show/123078051-eyeliner
29	link	/book/show/2767052-the-hunger-games
30	link	/book/show/10429045-shatter-me
31	link	/book/show/52867387-beach-read



DEVELOPMENT PHASE

**PHASE 1:
PROJECT PLANNING**

**PHASE 2:
ENVIRONMENT SETUP**

**PHASE 3:
BACKEND SCRAPING
ENGINE**

**PHASE 4:
FLASH BACKND API**

**PHASE 5:
FRONTEND
INTERFACE**

**PHASE 6:
SECURITY LAYER**

**PHASE 7:
TESTING AND VALIDATION**

**PHASE 8:
DOCUMENTATION**

PHASE 01: PROJECT PLANNING

Define exactly what the system will do before writing code.

Goal definiton → Architecture design → Feature planning → Role assignement

Define the main idea:

- Build an ethical web scraping system

Decide inputs:

- A website URL provided by user

Decide outputs:

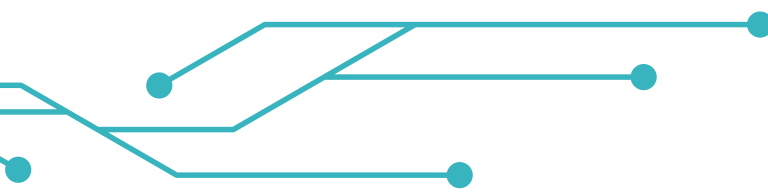
- Extracted structured data (text/links/images)
- Export formats: (CSV/JSON/Excel)

Designed system rules:

- Must respect website rules (robots.txt)
- Must avoid overloading websites (rate limiting)
- Must use secure connections (HTTPS/TLS)

Designed architecture:

- Frontend → Backend → Scraper → Output system



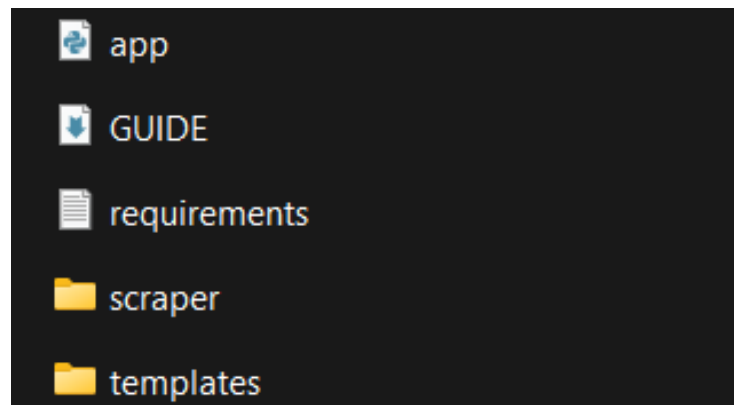
==> Define exactly what the system will do before writing code.

PHASE 02: ENVIRONMENT SETUP

Prepare everything needed to run the project.

Installed Python 3.9

Created project folder structure



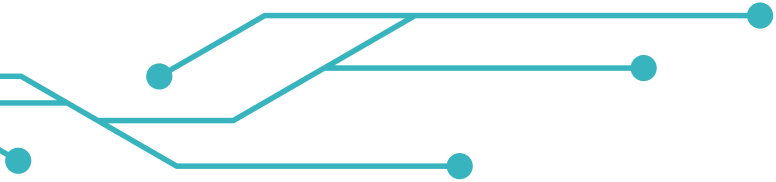
Created virtual environment (isolated workspace)

Installed dependencies

Libraries installed

- Flask → web server
- Requests → HTTP calls
- BeautifulSoup4 → HTML parsing
- lxml → faster parsing engine
- openpyxl → Excel file creation
- csv / json (built-in formats)

==> A fully ready development environment



PHASE 03: BACKEND SCRAPING ENGINE

Build the “brain” that extracts data from websites.



1/robots.txt check

- System first checks if website allows scraping
- Prevents illegal or blocked scraping

2/HTTP request (`fetch_html()`)

- Sends request to website using **Requests**
- Downloads raw HTML page

3/HTML parsing (`extract_data()`)

- Uses BeautifulSoup to read HTML
- Finds elements like:

<h1>, <h2> → titles

<p> → paragraphs

<a> → links

 → images

4/ Data transformation

Cleans data:

- removes empty values/duplicates
- trims spaces
- formats text

5/Save output (`save_output()`)

Stores data in different formats:

- CSV → table format
- JSON → structured format
- Excel → spreadsheet

==> Raw website → clean structured dataset

PHASE 04: FLASK BACKEND API

Connect scraper to a web system (make it usable like an app)



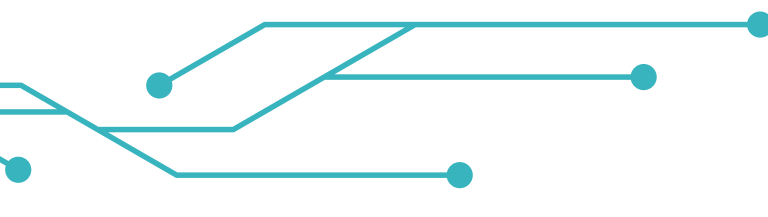
Flask acts as a communication bridge between frontend and backend.

API flow:

- 1/User sends URL from frontend
- 2/Flask receives request
- 3/Flask calls scraper engine
- 4/Scraper returns data
- 5/Flask sends response back to frontend

Endpoints:

- /
→ loads user interface (HTML page)
- **/api/scrape**
→ starts scraping process
- **/api/download**
→ sends output file



=> Your Python script becomes a web application

PHASE 05: FRONTEND INTERFACE

Create a simple interface for users.



What user can do :

- Enter a URL
- Choose data types:
text/links/images
- Select output format:
CSV / JSON / Excel
- Click "Scrape."
- See results
- Download file

How it works:

- Uses HTML for structure
- CSS for design
- JavaScript for logic
- Fetch API to communicate with Flask

=> User-friendly web interface



PHASE 06: SECURITY LAYER

Make scraping ethical and safe.



1. robots.txt compliance

- Checks if scraping is allowed before doing anything

2. Rate limiting

- Adds delay between requests
- Prevents server overload

3. HTTPS verification

- Ensures secure connection to website

4. User-Agent rotation

- Changes browser identity randomly
- Avoids bot detection

=> Safe + ethical scraping system



PHASE 07: TESTING AND VALIDATION

Ensure system works correctly in real conditions.



Websites used to test:

- books.toscrape.com
- https://coinmarketcap.com/?utm_source=chatgpt.com
- https://devpost.com/?utm_source=chatgpt.com

Error handling:

- invalid URL
- blocked site
- missing data

What was checked

- Does scraping extract correct data?
- Are files exported properly?
- Does API respond correctly?
- Does system block disallowed websites?

==> System is stable and reliable



PHASE 08: DOCUMENTATION

Explain how the system works for users and developers.



What was written

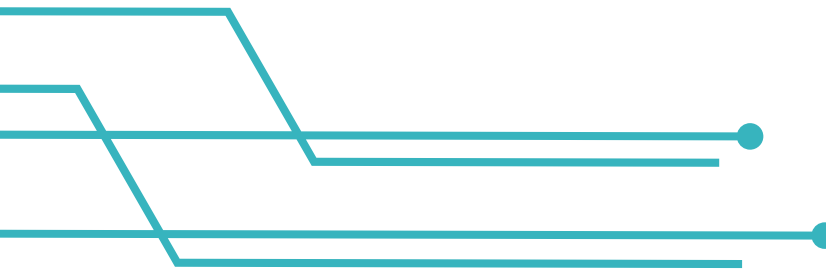
- How to install project
- How to run server
- How scraping works
- Security rules explained
- Example websites
- Common errors + fixes

==> Fully documented system (GUIDE.md)



TOOLS USED

Tool	Category	Purpose
 Python	Programming language	Writing scraping logic, processing data, saving files
 Flask	Web framework	Creating API, connecting frontend & backend, handling requests
 Requests	HTTP library	Opening websites and downloading HTML pages
 BeautifulSoup	HTML parser	Extracting data from websites (text, links, images)
 lxml	Fast parser	Making HTML parsing faster and more efficient
 openpyxl	Excel library	Creating Excel (.xlsx) files
 csv (stdlib)	Built-in Python tool	Saving data in CSV format
 json (stdlib)	Built-in Python tool	Saving structured data in JSON format
 HTML	Web structure	Building the user interface (pages, forms)
 CSS	Styling	Making the interface look good
 JavaScript	Frontend logic	Sending requests to Flask and showing results
 docx (Node.js)	Word generator	Creating formatted Word report automatically
 fs (Node.js)	File system tool	Saving the generated Word file on disk



IDENTIFIED THREATS

**IP BLOCKING /
BLACKLISTING**

**LEGAL
LIABILITY**

**MAN-IN-THE-
MIDDLE (MITM)
ATTACK**

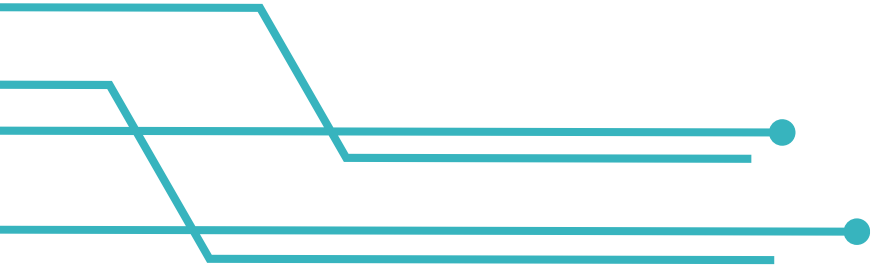
**BOT DETECTION
& ANTI-
SCRAPING**

**MALICIOUS
RESPONSE
INJECTION**

**DATA
POISONING**

**ACCIDENTAL
DENIAL OF
SERVICE (DOS)**

**SENSITIVE DATA
EXPOSURE**



HIGH RISK

IP Blocking / Blacklisting

The target server detects automated requests and blocks the scraper's IP address permanently or temporarily

Legal Liability

Scraping data from a website that explicitly prohibits it in its terms of service or robots.txt, leading to legal action

Accidental Denial of Service (DoS)

The scraper sends requests too quickly, overwhelming the target server and causing service degradation



MEDIUM RISK

Man-in-the-Middle (MITM) Attack

An attacker intercepts the HTTP communication between the scraper and the target website, stealing or modifying data

Malicious Response Injection

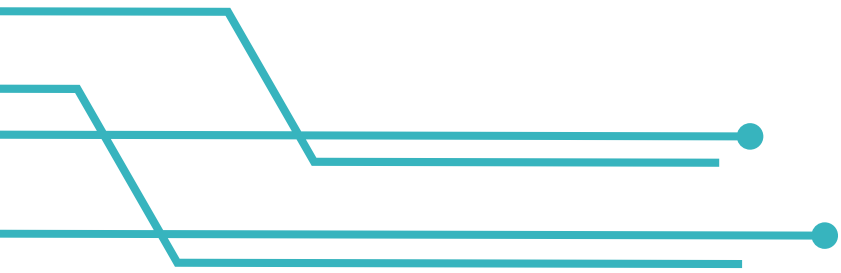
The server returns malicious content (e.g., JavaScript malware, XSS payloads) disguised as legitimate HTML

Bot Detection & Anti-Scraping

The website uses anti-bot techniques (honeypots, JavaScript challenges, rate monitoring) to identify and block scrapers

Sensitive Data Exposure

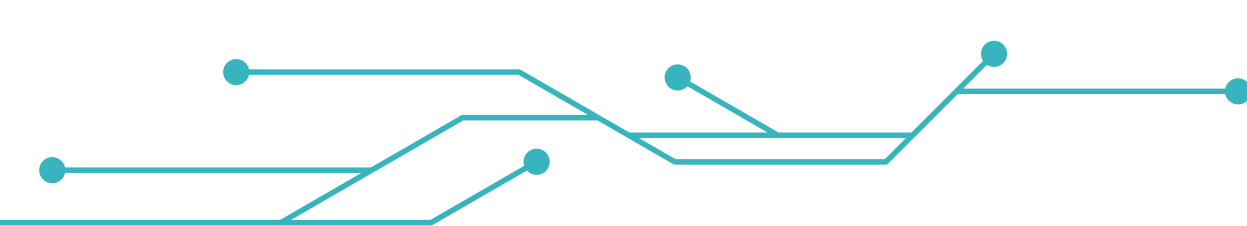
The scraper accidentally extracts and stores sensitive information (PII, credentials, internal URLs)



LOW RISK

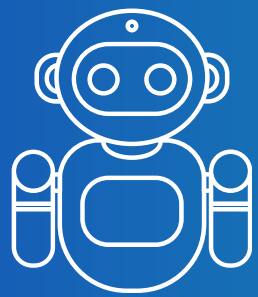
Data Poisoning

Website owners may intentionally insert fake data into their HTML to mislead scrapers. For example, a price comparison scraper might be fed incorrect prices, damaging the scraped results.



SECURITY MEASURES

To reduce these risks, several security mechanisms were integrated into the scraper.



Robots.txt Compliance

Automatic parsing of robots.txt to check website scraping permissions before any data extraction. If disallowed, the scraper aborts immediately.



Request Throttling

Configurable delays between requests (minimum 2 seconds) to prevent server overload and avoid IP blacklisting.



User-Agent Rotation

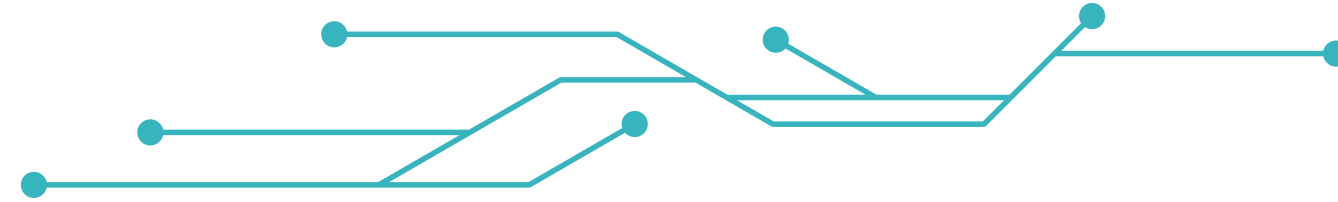
Random selection of real browser User-Agent strings for each request to avoid simple bot detection mechanisms.



TLS Encryption

Enforced HTTPS with certificate verification (verify=True) to secure all data transfers and prevent man-in-the-middle attacks.





Data Truncation

Automatic truncation of extracted data fields to maximum 1000 characters to prevent buffer overflow attacks.

Input Validation

Whitelist-based validation of all user inputs including URL format, field selection, and output format.

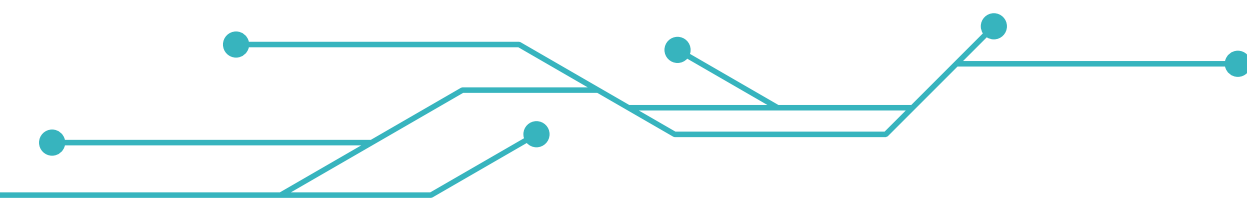
Safe Error Handling

Generic error messages returned to users without exposing internal file paths, stack traces, or system details

Structured Logging

Timestamped logging of all actions (successes and failures) to enable security audit trails and forensic analysis.





High Risk Threats

IP Blocking / Blacklisting	→	Request Throttling
Legal Liability	→	Robots.txt Compliance
Accidental Denial of Service (DoS)	→	Request Throttling

Medium Risk Threats

Man-in-the-Middle (MITM) Attack	→	TLS Encryption
Bot Detection & Anti-Scraping	→	User-Agent Rotation
Malicious Response Injection	→	Data Truncation
Sensitive Data Exposure	→	Input Validation

Low Risk Threats

Data Poisoning	→	Future Enhancement
----------------	---	--------------------

THANK YOU

